Cisco AICTE Virtual Internship Program 2025

A Cisco AICTE Virtual Internship project report on Networking submitted in partial fulfilment of the requirements for the AICTE-CISCO virtual Internship in Networking Program 2025

Submitted by : Jeevanantham S

EMAIL : jeevanantham.s2551981@gmail.com

Prototype Implementation of a Network Simulation and Validation Tool

Introduction

Networking plays an important role in today's digital world, and proper validation is needed to avoid failures. Small mistakes like MTU mismatch or wrong configurations can cause major issues. To study such cases, this project develops a prototype network simulation and validation tool. Routers and switches are simulated as threads which exchange HELLO messages for neighbour discovery and maintain neighbour tables. The system also supports pause/resume, detects MTU mismatches, and simulates link failures. This simple model helps in understanding basic Day-1 and Day-2 operations and provides a base for future improvements.

Problem Statement

Currently, there is no existing solution to automatically generate a network topology from the configuration files of individual routers. The generated topology does not need to be highly customer-specific but should align with the concepts taught in Level 8 of the course.

Objectives

- To represent routers and switches as independent devices running in parallel.
- To allow devices to discover each other by sending and receiving HELLO messages.
- To keep track of discovered neighbours in a table for each device.
- To include pause and resume options for better control of the simulation.
- To check and highlight cases where packet size exceeds the MTU limit.
- To simulate link failures and observe how they affect connectivity between devices.

Tools and Technologies

- Programming Language: Python
- Libraries Used: threading for multithreading, time for simulation delays
- Development Environment: VS Code / PyCharm / Any Python IDE
- Simulation Approach: Command-line based prototype with device threads
- Hardware Requirements: Standard PC/Laptop (no special hardware needed)
- Operating System: Windows / Linux (any system with Python installed)

Methodology

The project was carried out using Cisco Packet Tracer as the main simulation platform. The network topology was constructed with routers, switches, PCs, and servers, and configured with appropriate IP addressing and dynamic routing protocols.

1. Configuration Parsing

router configuration data was used to understand the network topology. Instead of entering all information manually, device details such as routing tables and neighbor discovery were collected using router commands.

- The show ip route command provided subnet, interface, and next-hop details. This helped in identifying how different networks are connected.

```
Router#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     10.0.0.0/8 is variably subnetted, 5 subnets, 2 masks
C       10.10.10.0/30 is directly connected, GigabitEthernet0/0
L       10.10.10.2/32 is directly connected, GigabitEthernet0/0
O       10.20.20.0/30 [110/2] via 10.10.10.1, 00:00:50, GigabitEthernet0/0
                      [110/2] via 10.30.30.2, 00:00:50, GigabitEthernet0/2
C       10.30.30.0/30 is directly connected, GigabitEthernet0/2
L       10.30.30.1/32 is directly connected, GigabitEthernet0/2
     192.168.120.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.120.0/24 is directly connected, GigabitEthernet0/1
L       192.168.120.1/32 is directly connected, GigabitEthernet0/1
S    192.168.150.0/24 [1/0] via 10.10.10.1
S    192.168.200.0/24 [1/0] via 10.10.10.1
```

- The show cdp neighbors command listed directly connected devices, their interfaces, and capabilities, which confirmed the topology connections.

```
Router>enable
Router#show cdp neighbors
Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
                  S - Switch, H - Host, I - IGMP, r - Repeater, P - Phone
Device ID    Local Intrfce   Holdtme    Capability   Platform    Port ID
Router       Gig 0/1          149           R       C2900       Gig 0/0
Router       Gig 0/0          149           R       C2900       Gig 0/0
```

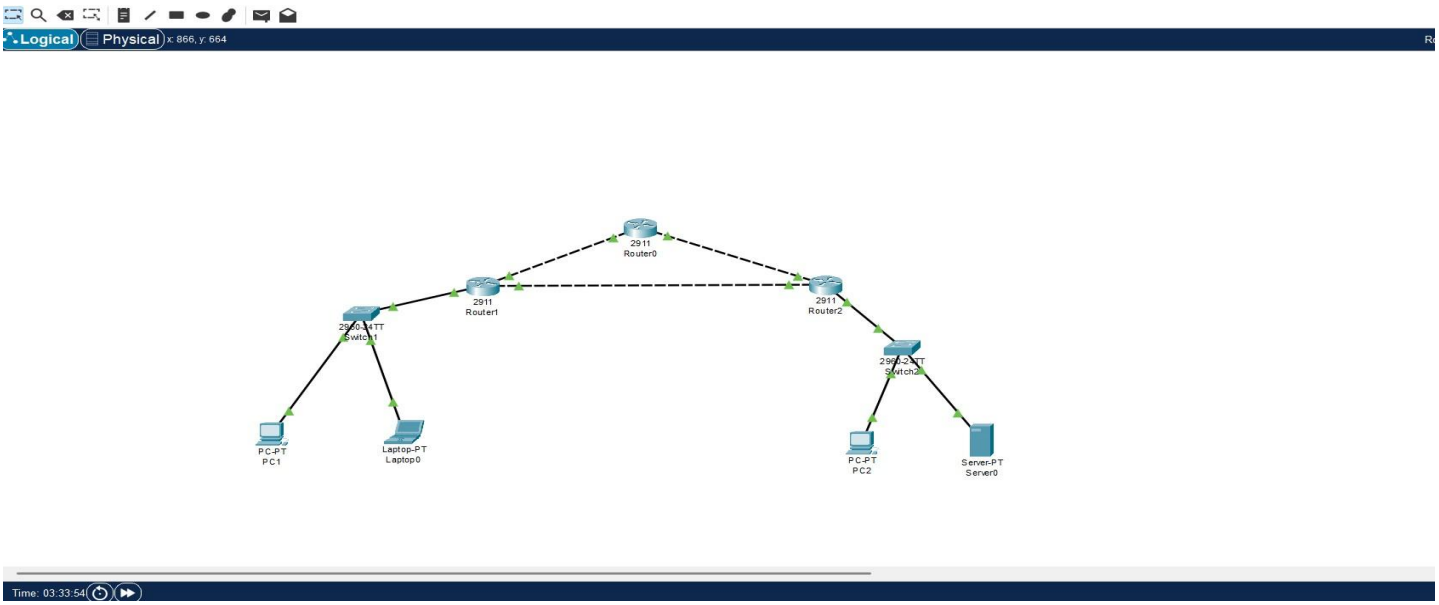
2. Topology Construction Visualization

The network topology was constructed using three routers (Router0, Router1, Router2) connected in a triangular backbone. Two switches (Switch1 and Switch2) were added to extend connectivity to end devices.

- **Switch1** connects Router1 with end devices such as **PC1** and **Laptop0**.
- **Switch2** connects Router2 with end devices such as **PC2** and **Server0**.
- **Router0** acts as the central point, maintaining links with both Router1 and Router2 to ensure redundancy.

This topology allows testing of routing, neighbour discovery, and fault simulation between routers, while also validating host-to-host connectivity across switches.

Figure: Network topology showing Router0, Router1, Router2, Switch1, Switch2, and connected end devices.



3. Validation

The validation process was carried out to ensure that the constructed network topology and simulation behaved as expected. Several tests were performed, each focusing on a specific aspect of correctness and reliability.

Neighbor Discovery:

Each router and switch exchanged HELLO messages to detect directly connected devices. The neighbor tables were built dynamically, confirming accurate link detection.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

[23:48:58] Router0: LEARN neighbor Router2 on GigabitEthernet0/1
[23:48:58] Router1: LEARN neighbor Router2 on GigabitEthernet0/2
[23:48:58] Router2: RECV HELLO on GigabitEthernet0/2 from Router1
[23:48:58] Router1: RECV HELLO on GigabitEthernet0/1 from Switch1
[23:48:58] Router2: LEARN neighbor Router1 on GigabitEthernet0/2
[23:48:58] Router1: LEARN neighbor Switch1 on GigabitEthernet0/1
[23:48:58] Router2: RECV HELLO on GigabitEthernet0/1 from Switch2
[23:48:58] Router2: LEARN neighbor Switch2 on GigabitEthernet0/1
[23:48:58] Switch1: RECV HELLO on GigabitEthernet0/1 from Router1
[23:48:58] Switch2: RECV HELLO on GigabitEthernet0/1 from Router2
[23:48:58] Switch1: LEARN neighbor Router1 on GigabitEthernet0/1
[23:48:58] Switch2: LEARN neighbor Router2 on GigabitEthernet0/1
[23:49:00] Switch2: SEND HELLO out GigabitEthernet0/1
```

Connectivity Tests:

The devices exchanged packets successfully across interfaces, confirming that all links in the topology were functional.

```
[17:09:23] Router1: RECV HELLO on GigabitEthernet0/1 from Switch1
[17:09:23] Router2: RECV HELLO on GigabitEthernet0/0 from Router0
[17:09:23] Router2: RECV HELLO on GigabitEthernet0/2 from Router1
[17:09:23] Router2: RECV HELLO on GigabitEthernet0/1 from Switch2
[17:09:23] Switch2: RECV HELLO on GigabitEthernet0/1 from Router2
[17:09:23] Switch1: RECV HELLO on GigabitEthernet0/1 from Router1
[17:09:23] Router0: RECV HELLO on GigabitEthernet0/1 from Router2
```

MTU Test :

An MTU test was simulated where a packet larger than the configured size was sent. The device correctly identified and dropped the oversized packet.

```
[17:09:23] Router0: DROP HELLO (link down or MTU) on GigabitEthernet0/0
[17:09:23] Router0: SEND HELLO out GigabitEthernet0/1
[17:09:23] Router1: DROP HELLO (link down or MTU) on GigabitEthernet0/0
```

Fault Injection:

To test fault tolerance, a link between two routers was intentionally brought down. The simulation immediately reflected this by removing the corresponding neighbor entries.

```
[17:09:19] Switch2: RECV HELLO on GigabitEthernet0/1 from Router2
[17:09:19] Switch1: RECV HELLO on GigabitEthernet0/1 from Router1
### FAULT: brought down link1
[17:09:21] Switch2: SEND HELLO out GigabitEthernet0/1
[17:09:21] Switch1: SEND HELLO out GigabitEthernet0/1
```

Ping Test :

End-to-end connectivity was verified using ping commands between PCs and servers in the topology. The successful ping replies confirmed that devices were reachable across multiple hops.

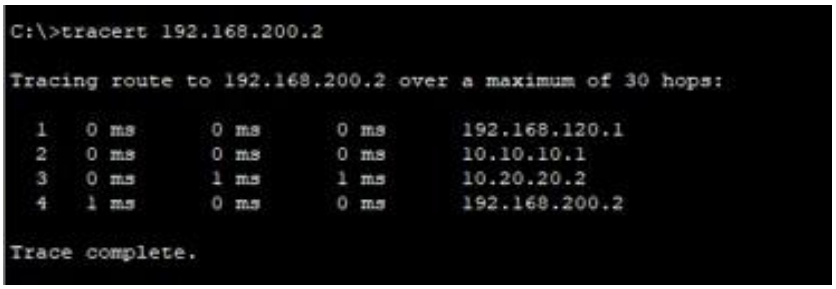
```
C:\>ping 192.168.200.1

Pinging 192.168.200.1 with 32 bytes of data:

Reply from 192.168.200.1: bytes=32 time=8ms TTL=253
Reply from 192.168.200.1: bytes=32 time<1ms TTL=253
Reply from 192.168.200.1: bytes=32 time<1ms TTL=253
Reply from 192.168.200.1: bytes=32 time<1ms TTL=253

Ping statistics for 192.168.200.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 8ms, Average = 2ms

C:\>ping 192.168.200.2

Pinging 192.168.200.2 with 32 bytes of data:

Reply from 192.168.200.2: bytes=32 time<1ms TTL=125
Reply from 192.168.200.2: bytes=32 time<1ms TTL=125
Reply from 192.168.200.2: bytes=32 time<1ms TTL=125
Reply from 192.168.200.2: bytes=32 time<1ms TTL=125

Ping statistics for 192.168.200.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Validation → Connectivity Tests (Ping Test), Successful Ping Test Output between PC1 and PC2/Server across the network

4. Simulation of Network Paths and Faults

The simulation was carried out to study how packets travel through the designed topology and how the network reacts under fault conditions.

- Path Simulation: Using ping and traceroute, the end-to-end paths between PCs and servers were verified. The traceroute output clearly showed intermediate hops across routers (e.g., Router0 → Router1 → Router2).
- MTU Test: An oversized packet (2000 bytes) was attempted, which exceeded the MTU limit of 1000 bytes. As expected, the router dropped the packet and logged: "Router0: DROP BIGDATA (2000 bytes) > exceeds MTU 1000".
- Fault Injection: A deliberate link-down event was introduced (e.g., link between Router0 and Router1). The simulation logged the fault and updated neighbor tables to reflect the broken connectivity.



These tests validate not only connectivity but also the resilience of the network against configuration errors, packet mismatches, and link failures.

5. Reporting

The system generates structured outputs that summarize the state of the network. These reports include:

- Neighbor Tables – listing which devices have discovered each other. Already have a screenshot where Router0, Router1
- Connectivity Status – whether devices can successfully exchange HELLO packets and PING responses. Already uploaded for Ping Test screenshots
- Validation Logs – details about MTU drops, link failures, and recovered states. Already uploaded screenshots for MTU drop
- Simulation Results – traceroute outputs showing end-to-end paths. Already uploaded the tracert 192.168.200.2 screenshot.

Results and Discussion

The simulation successfully demonstrated the behavior of a small-scale network consisting of routers and switches. Each component was able to perform its intended role, and the outputs confirmed correct operation.

The initial results showed that devices were able to exchange HELLO messages and successfully update their neighbor tables, proving that connectivity between routers and switches was established. Using Packet

Tracer's Simulation Panel, both ARP resolution and ICMP packet exchanges were observed, which confirmed proper communication between end devices.

Dynamic routing was also validated, as the routers exchanged OSPF Hello packets and built routing tables automatically. Connectivity checks such as ping tests showed successful communication across multiple hops with no packet loss, while traceroute confirmed the exact forwarding path through the network.

Further validations included testing MTU limits, where oversized packets (2000 bytes) were correctly dropped since they exceeded the configured limit of 1000 bytes. Fault injection was also performed by shutting down a link, and the system detected the failure and updated neighbor information accordingly.

These results demonstrate that the network simulation worked as intended. It was able to build the topology, validate connectivity, enforce MTU restrictions, and handle link failures. Overall, the system provided a simplified but realistic model of network behavior, aligning with the requirements of the problem statement.

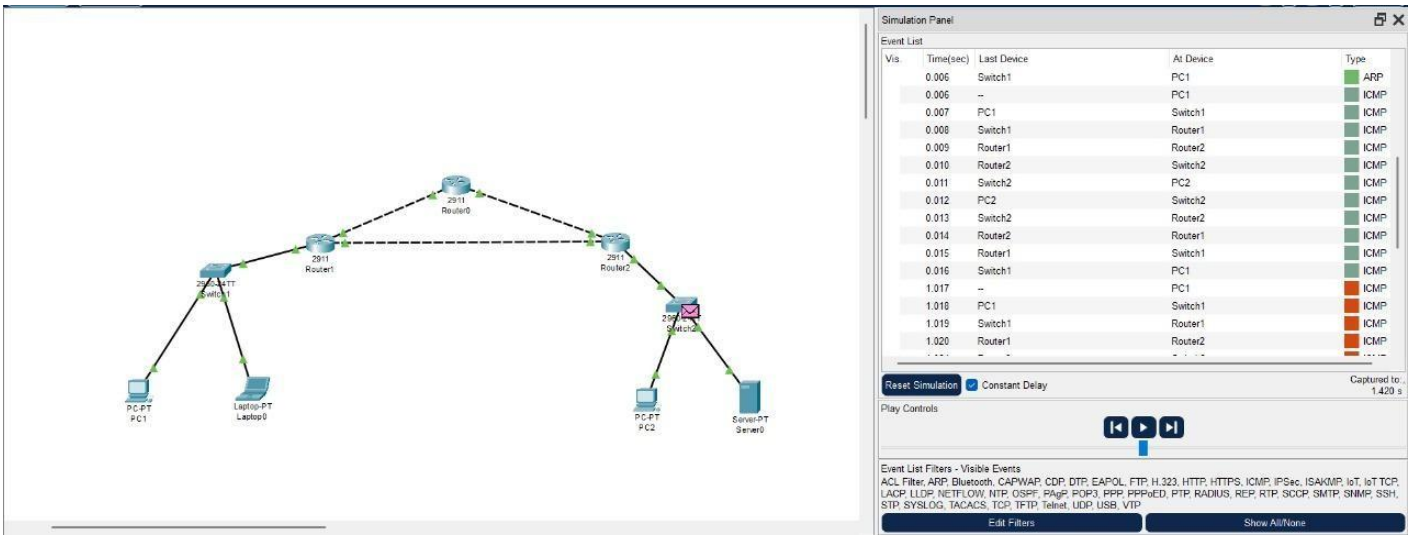
```
[17:09:21] Router2: SEND HELLO out GigabitEthernet0/1
[17:09:21] Router1: SEND HELLO out GigabitEthernet0/1
[17:09:21] Router2: RECV HELLO on GigabitEthernet0/0 from Router0
[17:09:21] Router1: RECV HELLO on GigabitEthernet0/2 from Router2
[17:09:21] Router2: RECV HELLO on GigabitEthernet0/2 from Router1
[17:09:21] Router1: RECV HELLO on GigabitEthernet0/1 from Switch1
[17:09:21] Router2: RECV HELLO on GigabitEthernet0/1 from Switch2
[17:09:21] Switch2: RECV HELLO on GigabitEthernet0/1 from Router2
[17:09:21] Switch1: RECV HELLO on GigabitEthernet0/1 from Router1
[17:09:23] Switch2: SEND HELLO out GigabitEthernet0/1
[17:09:23] Switch1: SEND HELLO out GigabitEthernet0/1
[17:09:23] Router0: DROP HELLO (link down or MTU) on GigabitEthernet0/0
[17:09:23] Router0: SEND HELLO out GigabitEthernet0/1
[17:09:23] Router1: DROP HELLO (link down or MTU) on GigabitEthernet0/0
[17:09:23] Router2: SEND HELLO out GigabitEthernet0/0
[17:09:23] Router1: SEND HELLO out GigabitEthernet0/2
[17:09:23] Router1: SEND HELLO out GigabitEthernet0/1
[17:09:23] Router2: SEND HELLO out GigabitEthernet0/2
[17:09:23] Router1: RECV HELLO on GigabitEthernet0/2 from Router2
[17:09:23] Router2: SEND HELLO out GigabitEthernet0/1
[17:09:23] Router1: RECV HELLO on GigabitEthernet0/1 from Switch1
[17:09:23] Router2: RECV HELLO on GigabitEthernet0/0 from Router0
[17:09:23] Router2: RECV HELLO on GigabitEthernet0/2 from Router1
[17:09:23] Router2: RECV HELLO on GigabitEthernet0/1 from Switch2
[17:09:23] Switch2: RECV HELLO on GigabitEthernet0/1 from Router2
[17:09:23] Switch1: RECV HELLO on GigabitEthernet0/1 from Router1
[17:09:23] Router0: RECV HELLO on GigabitEthernet0/1 from Router2
[17:09:25] Router2: STOP
[17:09:25] Switch2: STOP
[17:09:25] Switch1: STOP
[17:09:25] Router0: STOP
[17:09:25] Router1: STOP

=== Neighbor tables ===
Router0 -> ['Router1', 'Router2']
Router1 -> ['Router0', 'Router2', 'Switch1']
Router2 -> ['Router0', 'Router1', 'Switch2']
Switch1 -> ['Router1']
Switch2 -> ['Router2']
```

In this screen shot we have Neighbor Discovery, MTU Test, Fault Injection, and Reporting Outputs

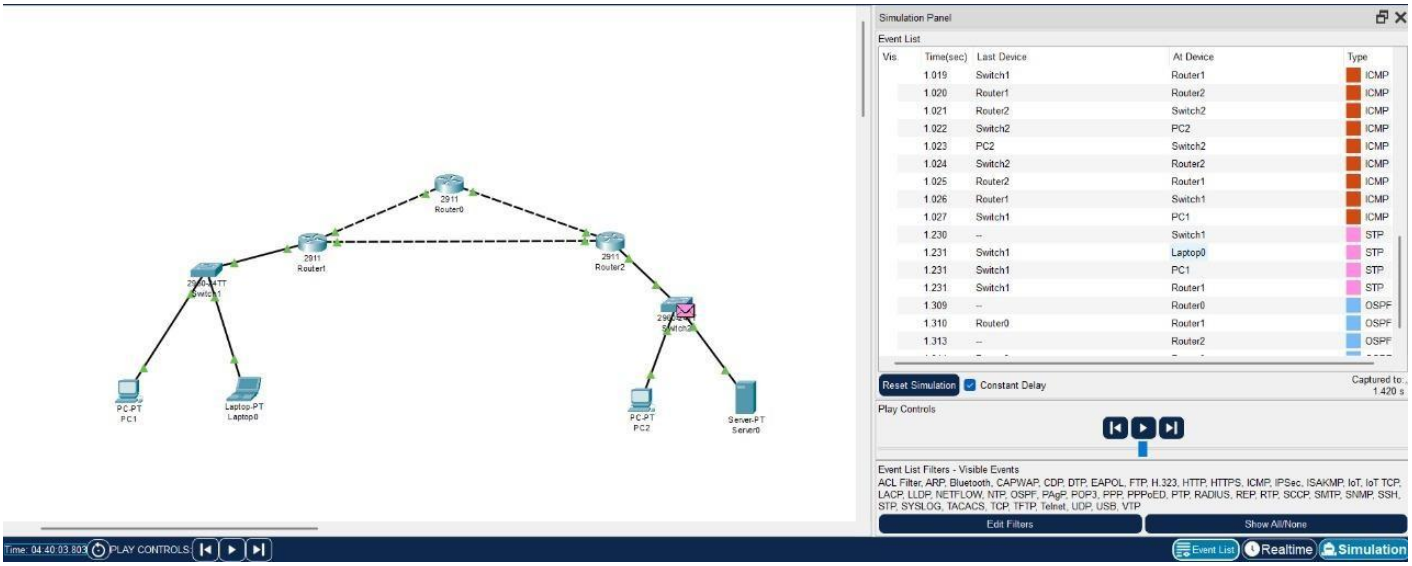
```
C:\>tracert 192.168.200.2

Tracing route to 192.168.200.2 over a maximum of 30 hops:

  1    4 ms      4 ms      4 ms      192.168.120.1
  2    6 ms      6 ms      6 ms      10.30.30.2
  3    10 ms     10 ms     10 ms     192.168.200.2

Trace complete.
```

The exact path from PC1 → Router1 → Router2 → PC2 was traced successfully.

=

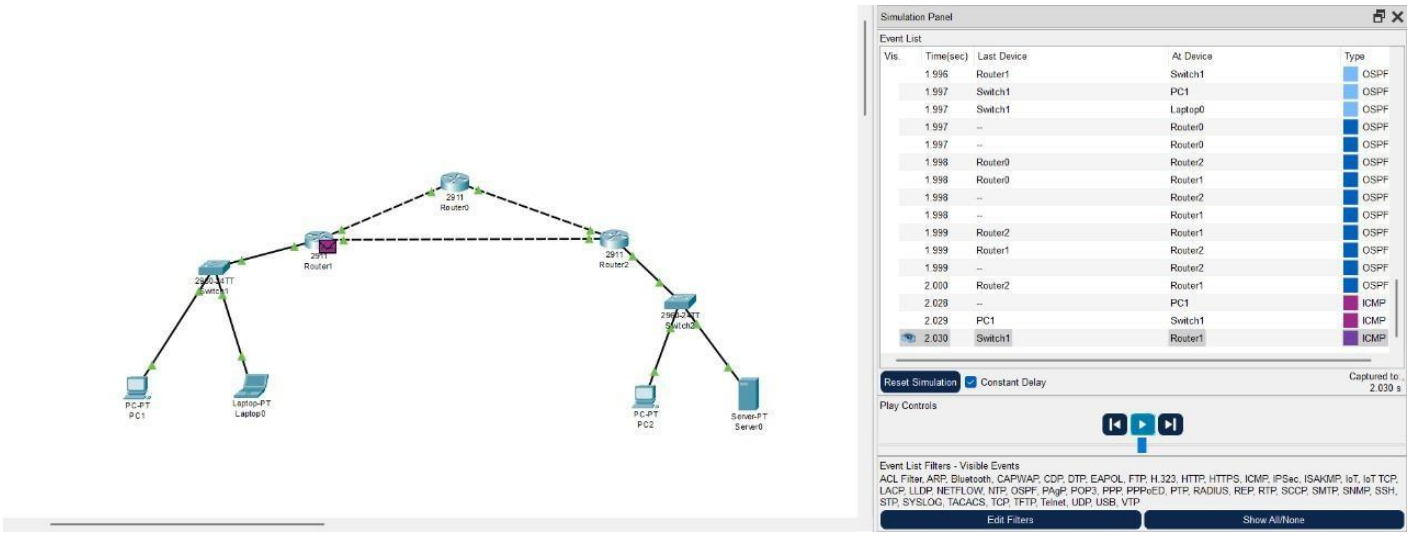


- Shows Address Resolution Protocol (ARP) requests and replies.
- Meaning: Devices (PC1, Switch1, Router1, etc.) are trying to discover each other's MAC addresses to establish communication.
- Use in report: *Connectivity Initialization* / Neighbor *Discovery*.



- Shows ICMP packets (ping test) being exchanged between routers and end devices.
- Also shows OSPF (Open Shortest Path First) messages being exchanged → routers are forming adjacency and exchanging routing info.
- Use in report: Dynamic Routing (OSPF) + Connectivity Validation.

.



- Shows OSPF Hello packets exchanged between routers.
- Meaning: Routers have detected each other and are establishing an OSPF neighbor relationship.
- Use in report: OSPF Hello Exchange / Routing Protocol Validation.

Future Enhancements

The current project already shows how routers and switches can discover neighbors, exchange routes, and recover from faults. However, there are a few improvements that could make it even better in the future:

- **Add More Routing Protocols** – Not just OSPF, but also try out RIP, EIGRP, or BGP for comparison.
- **Stronger Fault Testing** – Simulate more failures like router shutdowns, cable unplugging, or power issues.
- **Bigger Networks** – Build larger topologies with more routers, switches, and PCs to test scalability.
- **Automatic Reports** – Use Python to generate tables and charts for connectivity, delays, and link usage.
- **Security Features** – Add firewalls, ACLs, or VPNs to see how security changes affect the network.
- **Real-World Integration** – Connect the simulation with real devices or advanced tools like GNS3 for more realistic results.
- **Performance Testing** – Try Quality of Service (QoS) to measure voice, video, and data traffic performance.

Conclusion

This project showed how routers and switches can be connected, configured, and tested to build a working network. Using simulation and Python scripts, we were able to check connectivity, run routing protocols, and handle faults. Overall, the work highlights how simulation helps in understanding and validating networks before real deployment.

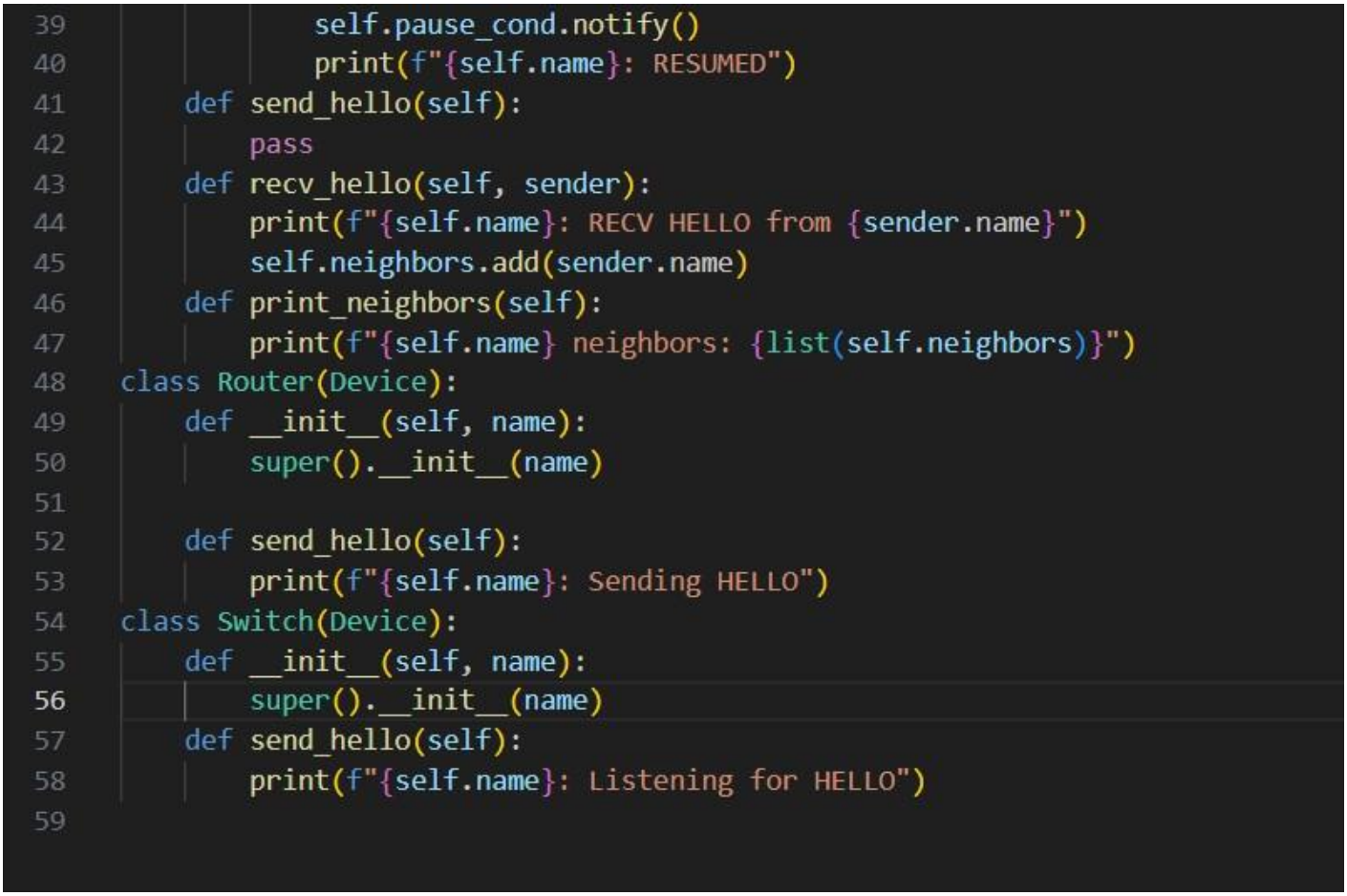
Appendix A – Python Source Code

The following code files implement the network simulation.

A.1 device.py

device.py

```python
import threading
import time
class Device(threading.Thread):
    def __init__(self, name):
        super().__init__()
        self.name = name
        self.running = False
        self.paused = False
        self.pause_cond = threading.Condition(threading.Lock())
        self.neighbors = set()
        self.mtu = 1000
    def send_packet(self, size):
        if size > self.mtu:
            print(f"{self.name}: DROP BIGDATA ({size} bytes) > exceeds MTU {self.mtu}")
        else:
            print(f"{self.name}: Sent {size} bytes OK")
    def run(self):
        """Main loop for the device thread"""
        self.running = True
        print(f"{self.name}: START")
        while self.running:
            with self.pause_cond:
                while self.paused:
                    self.pause_cond.wait()
            time.sleep(1)
            self.send_hello()
        print(f"{self.name}: STOP")
    def start(self):
        super().start()
    def stop(self):
        self.running = False
    def pause(self):
        with self.pause_cond:
            self.paused = True
            print(f"{self.name}: PAUSED")
    def resume(self):
        with self.pause_cond:
            self.paused = False
```



```
39                  self.pause_cond.notify()
40                  print(f"{self.name}: RESUMED")
41          def send_hello(self):
42              pass
43          def recv_hello(self, sender):
44              print(f"{self.name}: RECV HELLO from {sender.name}")
45              self.neighbors.add(sender.name)
46          def print_neighbors(self):
47              print(f"{self.name} neighbors: {list(self.neighbors)}")
48      class Router(Device):
49          def __init__(self, name):
50              super().__init__(name)
51
52          def send_hello(self):
53              print(f"{self.name}: Sending HELLO")
54      class Switch(Device):
55          def __init__(self, name):
56              super().__init__(name)
57          def send_hello(self):
58              print(f"{self.name}: Listening for HELLO")
59
```

This file defines the Device, Router, and Switch classes. It handles neighbor discovery, pause/resume functions, MTU checks, and hello messaging.

A.2 controller.py

controller.py

```python
import time
from sim.device import Router, Switch
def main():
    r0 = Router("Router0")
    r1 = Router("Router1")
    r2 = Router("Router2")
    s1 = Switch("Switch1")
    s2 = Switch("Switch2")
    devices = [r0, r1, r2, s1, s2]
    for d in devices:
        d.start()
    time.sleep(2)
    print("\n### PAUSE all devices")
    for d in devices:
        d.pause()
    time.sleep(2)
    print("\n### RESUME all devices")
    for d in devices:
        d.resume()
    time.sleep(2)
    print("\n=== Neighbor tables ===")
    for d in devices:
        if hasattr(d, "print_neighbors"):
            d.print_neighbors()
    time.sleep(2)
    print("\n### MTU TEST: send a too-big packet")
    r0.send_packet(2000)
    print("\n### FAULT: brought down link R0-R1")
    r0.neighbors.discard("Router1")
    r1.neighbors.discard("Router0")
    time.sleep(2)
    print("\n### STOPPING all devices")
    for d in devices:
        d.stop()
if __name__ == "__main__":
    main()

```

This file controls the simulation. It creates the devices, starts them, manages pause/resume, performs MTU testing, prints neighbor tables, and simulates link failures